Università degli studi di Ferrara
Dipartimento di Ingegneria
CdLM Ingegneria Informatica e dell'Automazione

Laboratorio di Industria 4.0
Progetto: Smart Warehouse

Progettazione e implementazione di un'architettura event-driven
per la gestione logistica

Docente
Prof. Mauro Tortonesi

Studente
Elia Felletti

Anno Accademico 2025/2026

Indice

Indice **2**
1. Introduzione e obiettivi del progetto **3**
1.1 Contesto operativo 3
1.2 Requisiti funzionali 3
2. Architettura della Pipeline Big Data **4**
3. Scelte tecnologiche e motivazioni **7**
3.1 Apache Kafka in modalità KRaft 7
3.2 Adozione dell'ecosistema confluent (Docker images) 8
3.3 Confluent REST proxy 9
3.4 PostgreSQL 10
3.5 Kafka Connect JDBC Sink 12
3.6 Dashboard Grafana 14
3.7 Strategia di delivery: at-least-once 14
4. Modello dei dati **16**
5. Implementazione dei servizi custom **17**
5.1 Emulazione del terminale mobile 17
5.2 Elaborazione degli eventi - core business logic 20
6. Orchestrazione tramite Docker Compose **23**
7. Conclusioni e sviluppi futuri **25**
Appendice **27**

1. Introduzione e obiettivi del progetto

1.1 Contesto operativo

Uno smart warehouse è un edificio adibito allo **stoccaggio di materie prime o semilavorati** che si avvale dell'integrazione tra macchine e sistemi informatici per automatizzare le operazioni di magazzino tradizionalmente svolte dall'uomo. Queste attività comprendono l'identificazione e la ricezione degli ordini, il conteggio e la gestione dei prodotti, il controllo degli accessi ai locali e il trasferimento fisico dei materiali tra i diversi settori della struttura. L'obiettivo primario di tale evoluzione tecnologica è l'**ottimizzazione dell'efficienza operativa e la riduzione del margine di errore umano** nella gestione logistica.

1.2 Requisiti funzionali

Lo scopo del presente progetto è la realizzazione di un sistema integrato per la gestione di uno smart warehouse di un'industria manifatturiera, in particolare il sistema è stato pensato per soddisfare i seguenti requisiti funzionali:

1. **controllo degli accessi**: garantire l'accesso a determinati locali solo al personale che ne ha effettivamente i permessi, attraverso un controllo del badge personale;
2. **gestione dinamica dell'inventario**: aggiornamento dinamico e immediato di tutte le giacenze di magazzino, a seguito di ogni prelievo o deposito;
3. **automatizzazione riordini**: in aggiunta agli ordini manuali inseribili nel sistema in qualsiasi momento (per ordini espliciti dei clienti), nel momento in cui la giacenza di un prodotto vada al di sotto di una certa soglia si innesca il riordino automatico della merce;

4. **reportistica e monitoraggio**: attraverso una dashboard aggiornata in tempo reale, viene fornita una visione d'insieme del magazzino completa di statistiche storiche e in tempo reale.

2. Architettura della Pipeline Big Data

L'intera architettura è stata progettata come **pipeline** end-to-end per garantire flussi di informazione continui e persistenti. Tale pipeline si articola di 5 layer fondamentali, ognuno con compiti e task specifici nel ciclo di vita del dato.
Il punto centrale dell'architettura è il messaging layer dove si trova Apache **Kafka**, broker di messaggi per favorire comunicazione asincrona e disaccoppiamento tra i vari servizi. Sono stati predisposti 3 topic:

- stock movements: registra tutti gli eventi di movimentazione della merce, sia per i depositi che per i prelievi;
- orders: registra tutti i riordini della merce, sia automatici che manuali;
- access events: registra tutti i tentativi di accesso ai settori del magazzino, sia consentiti che negati.

Il flusso dei dati parte dall'ingestion layer, che si occupa di raccogliere i dati provenienti dal magazzino e immetterli nel sistema, più nello specifico va a raccogliere le operazioni che gli operatori svolgono dal palmare in loro dotazione. Per il layer in questione è stato scelto un **proxy REST** che espone una **API HTTP standard**, attraverso il quale vengono inseriti i messaggi generati all'interno dei rispettivi topic Kafka.
Successivamente, per quanto riguarda lo storage layer, la persistenza di tutti i dati è affidata al DBMS relazionale **PostgreSQL**, che funge da *Single Source of Truth*, all'interno del database troviamo le informazioni riguardanti: prodotti, settori, dati anagrafici degli utenti, stato corrente del magazzino con tutte le giacenze, storico delle movimentazioni, storico degli accessi ai settori del

magazzino e indicazioni riguardanti le autorizzazioni dei singoli utenti nell'accesso ai settori del magazzino.

Il processamento di tutte le operazioni, definito nel processing layer, presenta due componenti principali:

- business logic: si occupa di elaborare tutte le movimentazioni merce e aggiornare immediatamente l'inventario tramite operazioni di UPDATE o UPSERT, inoltre, a seguito di ogni prelievo controlla che la quantità non sia scesa sotto la soglia e, in tal caso, procede al riordino automatico (**custom logic consumer** realizzato in **python**);
- data integration: per la sola storicizzazione all'interno del database di tutti gli eventi che vengono pubblicati sui topic Kafka, si è scelto di adottare **Kafka Connect** per automatizzare il tutto. Tale componente permette di avere una perfetta sincronizzazione tra messaging layer e storage layer.

Come ultimo si trova il serving layer, con lo scopo di visualizzare e analizzare i dati per fornire supporto decisionale al management. Lo strumento scelto è **Grafana** che, grazie alla costruzione di una dashboard interattiva, permette di monitorare in tempo reale i principali dati e KPI del magazzino. In questo layer si vanno a manipolare i dati grezzi presenti nelle tabelle del database e li si trasformano in intuitivi pannelli. La dashboard di riferimento è suddivisa in 3 parti:

- statistiche globali: panoramica istantanea dello stato del magazzino nella quale si trovano conteggio generale di tutte le merci, prodotti momentaneamente al di sotto della soglia del riordino, elenco completo delle ultime movimentazioni e dei riordini e un'ultima sezione riguardante gli accessi negati;
- dettaglio settore: grazie alla variabile dinamica predisposta è possibile visualizzare le merci presenti e le movimentazioni relative al settore selezionato;

- dettaglio prodotto: grazie alla variabile dinamica predisposta è possibile visualizzare la distribuzione del prodotto tra i vari settori (pie chart), lo storico movimenti specifico e pannelli statistici che confrontano stock attuale e soglia di riordino.

Tale suddivisione in layer della pipeline permette di seguire ogni fase del ciclo di vita del dato, dalla sua generazione fino alla dashboard finale. L'architettura realizzata consente di monitorare in real time il magazzino, soddisfacendo nella loro totalità i requisiti di monitoraggio e automazione del progetto.

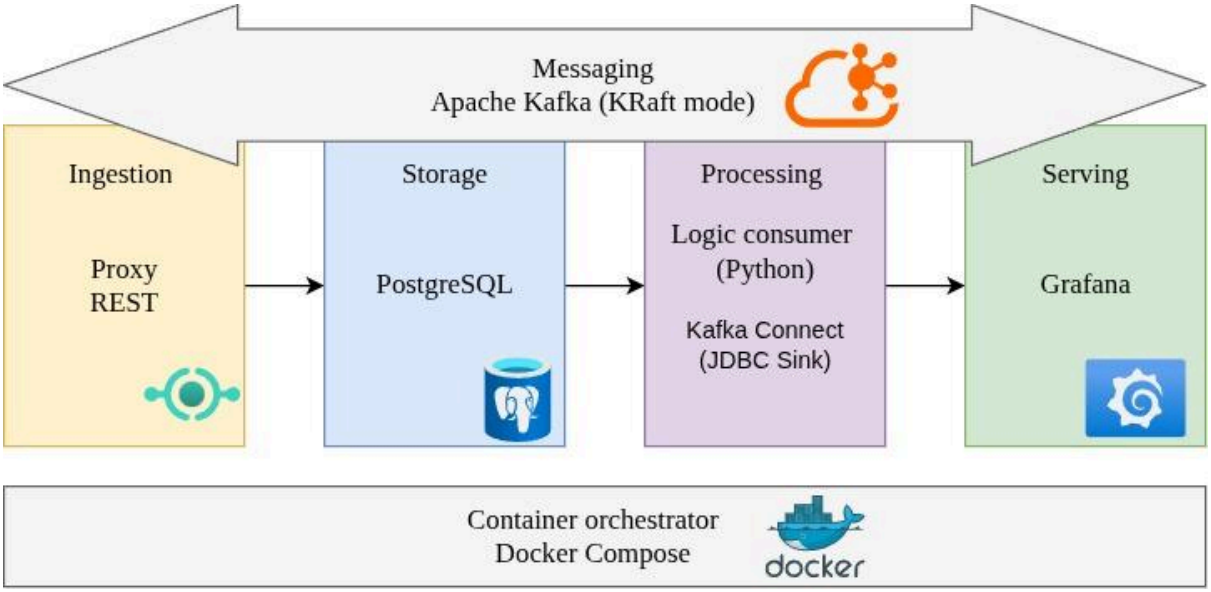

Figura 2.1 Layer della Big Data pipeline

3. Scelte tecnologiche e motivazioni

In questo capitolo vengono analizzate le principali **scelte progettuali**, con relative **motivazioni**, che hanno guidato lo sviluppo dell'intero progetto. Ogni componente dello stack tecnologico è stato scelto con l'obiettivo di rispettare tutti i requisiti tipici di una Big Data pipeline.

3.1 Apache Kafka in modalità KRaft

La scelta tecnologica fondamentale risiede nell'adozione di **Apache Kafka con abilitata la modalità KRaft** al fine di gestire il cluster. Storicamente, un cluster Kafka richiedeva la presenza di un coordinatore terzo, ovvero ZooKeeper, una dipendenza esterna che si incaricava di gestire i metadati, l'elezione del nodo leader e la configurazione dei topic. Nel seguente progetto si è scelto di superare questa architettura tradizionale, a favore di questo nuovo protocollo di consenso nativo, per varie motivazioni:

- **gestione del Control Plane semplificata**: la scelta di abilitare KRaft permette di aggregare i metadati all'interno di un log gestito tramite un algoritmo di consenso basato su Raft, invece di avere due sistemi separati con necessità di sincronizzazione continua. Questa scelta va ad eliminare il potenziale problema del "disallineamento" dei metadati in cui broker e controller hanno due visioni differenti dello stato del sistema;
- **ruolo misto** del nodo Kafka: all'interno del file di orchestrazione dei servizi, il parametro `KAFKA_PROCESS_ROLES: broker,controller` permette al nodo di effettuare il dispatch dei messaggi (*Data Plane*) e gestore del quorum dei metadati (*Control Plane*). Ciò riduce notevolmente l'overhead di rete e di memoria nella gestione dell'infrastruttura;

- velocità di recovery: nei sistemi basati su ZooKeeper in caso di guasto i tempi necessari per l'elezione di un nuovo leader possono richiedere qualche secondo, mentre con KRaft l'elezione del nuovo leader è quasi istantanea poiché lo stato del controller è replicato e pronto nei nodi Kafka. Nella gestione di un magazzino dove la discrepanza tra giacenze reali e a inventario può causare perdita di tracciabilità è fondamentale un veloce tempo di recovery.

In sintesi, l'uso di KRaft converte il solo broker di messaggi in un sistema auto-contenuto più robusto, garantendo che il messaging layer sia solido per tutta la pipeline.

3.2 Adozione dell'ecosistema confluent (Docker images)

Durante la fase di progettazione dell'infrastruttura si è scelto di adottare l'ecosistema di immagini Docker fornito da Confluent (*cp-kafka*, *cp-kafka-rest*, *cp-kafka-connect*).

Nonostante la natura open-source di Apache Kafka, tale ecosistema (confluent platform) è stato scelto per ragioni di **omogeneità**, **stabilità** e **integrazione**, al fine di rimuovere eventuali conflitti di versione o incompatibilità nei protocolli di comunicazione binaria. Questa decisione permette al broker di dialogare in modo trasparente con il REST proxy e con il framework Connect, assicurando che la **serializzazione** sia **coerente lungo tutto il percorso del dato**.

Sotto l'aspetto dell'efficienza, un ulteriore vantaggio che offrono le immagini Confluent consiste nel pre-caricamento di numerosi plugin e librerie per ottimizzare una pipeline Big Data. Ad esempio, il servizio Connect include nativamente la gestione avanzata di JSON con schemi, facilitando l'integrazione del JDBC Sink Connector verso il layer di storage. Così facendo viene ridotto drasticamente il tempo necessario alla configurazione o all'installazione di

moduli in modo manuale, consentendo di focalizzarsi sulla logica di mappatura dei dati e sulla persistenza.
Infine, l'omogeneità si ripercuote positivamente anche nel file *compose* di configurazione di tutti i servizi: la standardizzazione delle variabili d'ambiente adottata da Confluent (prefissi di esempio: `KAFKA_`, `CONNECT_` e `KAFKA_REST_`) rende tale file molto più leggibile, ordinato, modulare e mantenibile.
In sintesi, l'ecosistema Confluent rende la pipeline fortemente coerente e resiliente, minimizzando le criticità operative e assicurando integrità dei dati tra i vari servizi.

3.3 Confluent REST proxy

Per la fase di ingestion delle operazioni di magazzino si è scelto di impiegare il Confluent REST proxy come interfaccia di ingresso verso il cluster Kafka. Tale decisione è motivata dalla volontà di **semplificare il progetto del client**, rendendolo più "leggero", per permettergli di interagire con complessi sistemi di messaggistica **senza introdurre overhead** computazionale o dipendenze tecnologiche vincolanti.
L'utilizzo dell'approccio architetturale REST consente di astrarre dal protocollo binario di Kafka il quale, pur essendo più performante, richiede che il client si incarichi della gestione di connessione TCP persistenti, metadati e partizionamento dei messaggi. Si osserva che imporre tutti i sopra citati compiti a un palmare in dotazione all'operatore di magazzino sia del tutto inefficiente. La scelta di esporre una API HTTP standard trasforma il palmare in un *thin-client* che si limita ad inviare delle HTTP request con metodo POST in formato JSON. Così facendo lo sviluppo del software lato client è semplificato e si rende l'infrastruttura pronta ad accogliere una moltitudine di sorgenti di informazioni, siano esse sensori IoT o applicazioni web, senza problematiche

legate alla connessione mantenendo interoperabilità e indipendenza dal linguaggio di programmazione o dalla piattaforma.
Inoltre, il proxy agisce da layer di **isolamento e sicurezza** per il sistema di messaggistica. In ambito di produzione, esporre **direttamente il broker di messaggi alle sorgenti dati comporterebbe rischi legati alla sicurezza** e vulnerabilità sfruttabili da potenziali attaccanti. Centralizzando il punto di ingresso delle informazioni è possibile applicare logiche di load balancing, autenticazione e monitoraggio, con l'obiettivo di proteggere l'integrità, la riservatezza e la disponibilità di tutti i dati circolanti nel sistema.
In riferimento al contesto del progetto, è stato necessario strutturare i messaggi in modo autodescrittivo tramite l'header `Content-Type: application/vnd.kafka.json.v2+json`, ciò assicura che ogni messaggio venga consegnato ai topic con la corretta formattazione per le fasi di processing successive.
Come ultimo aspetto, dato che il REST proxy è un componente dell'architettura completamente stateless abilita una forte **scalabilità orizzontale**. Al crescere del volume del traffico e delle richieste provenienti da centinaia di operatori simultaneamente, è possibile istanziare facilmente delle repliche del proxy per non intaccare performance e stabilità del sistema.
In sintesi, si va a trasformare il processo di acquisizione dei dati in un servizio flessibile e standardizzato, rispettando le esigenze di mobilità e leggerezza del client.

3.4 PostgreSQL

Per lo storage la scelta tecnologica è ricaduta sul conosciutissimo PostgreSQL, in particolare nella sua versione 13. Il ruolo ricoperto da tale tecnologia non è di semplice archiviazione, ma riveste il ruolo critico di *Single Source of Truth* (sorgente unica delle verità), centralizzando dati anagrafici, lo stato dinamico

dell'inventario e lo storico immutabile delle movimentazioni o riordini. Inoltre, viene applicato come sorgente dati per il serving layer, soprattutto perché la potenza del linguaggio SQL permette di eseguire query complesse e aggregazioni in tempo reale.

Le motivazioni che hanno portato alla scelta di un DBMS relazionale, rispetto ad una alternativa NoSQL, sono strettamente legate alla natura del dominio applicativo:

- **ACID e integrità transazionale**: in un contesto applicativo in cui le approssimazioni sulle giacenze di magazzino non sono assolutamente ammesse, le proprietà ACID (Atomicità, Consistenza, Isolamento e Durabilità) assicurano che ogni operazione, sia essa di deposito o prelievo, venga registrata in modo atomico. Ciò previene perdite di dati, di conseguenza perdita di tracciabilità della merce, a causa di crash del sistema durante la fase di aggiornamento dell'inventario. In questo modo si ha la garanzia di allineamento tra informazioni sul database e realtà fisica del magazzino;
- **vincoli di integrità**: PostgreSQL consente di inserire dei vincoli fisici rigorosi direttamente nello schema del database. Ad esempio, le chiavi esterne che assicurano movimentazioni riferite a prodotti o settori effettivamente esistenti oppure le clausole CHECK per effettuare controlli aggiuntivi al valore di specifiche colonne delle tabelle (nella tabella Inventory è stato applicato check quantity >=0 per le giacenze);
- supporto alla **semantica "at-least-once"**: qui si va a sfruttare l'apertura di PostgreSQL a transazioni esplicite, il database consente al *Logic Consumer* di coordinare l'aggiornamento dello stato con i messaggi presenti nei topic Kafka. Solo quando il database confermerà di aver scritto correttamente tutte le modifiche, il sistema procederà all'avanzamento dell'offset, eliminando il rischio di inconsistenze tra messaging layer e storage layer.

In sintesi, PostgreSQL è stato scelto non solo per la sua forte stabilità, ma come uno strumento di data governance attivo, in grado di imporre ordine e rigore logico a flussi di dati.

3.5 Kafka Connect JDBC Sink

Per assicurare la persistenza di tutte le operazioni che transitano attraverso Kafka, è stato integrato il framework Kafka Connect, utilizzando nello specifico il JDBC Sink Connector. Qui è osservabile la separazione netta tra operazioni di business complesse (Logic Consumer analizzato nel capitolo 5) e semplici trasferimenti massivi di informazioni dai topic verso lo storage layer. E' importante sottolineare che la configurazione dei connettori non avviene attraverso file statici, bensì avviene in modo dinamico grazie alle API REST native solitamente esposte sulla porta 8083 (si veda esempio a fine sezione).
Le motivazioni alla base di questa scelta includono:

- **riduzione del codice e manutenibilità**: non è necessario scrivere ulteriore codice "boilerplate" per interfacciare manualmente il messaging layer con lo storage layer. Inoltre, questo framework si incarica di aspetti critici come l'apertura della connessione, l'esecuzione delle INSERT e la corrispondenza dei tipi di dato. Il sistema è così più manutenibile dato che una modifica alle tabelle dei log nel database è risolta con una sola modifica al JSON di configurazione dei connettori;
- **JSON schema e mapping dinamico**: il connettore è in grado di mappare autonomamente i campi del payload Kafka con le colonne delle tabelle del database, tale vantaggio è dovuto all'adozione dell'ecosistema Confluent. Tra le configurazioni è stato scelto di inserire l'opzione `"auto.create": "false"`, per evitare che il framework Connect abbia la facoltà di alterare lo schema del database. Di conseguenza, le tabelle in cui andrà ad inserire i dati dovranno essere già esistenti e

perfettamente corrispondenti ai valori presenti nei messaggi provenienti dal topic, in caso di non perfetta corrispondenza l'inserimento verrà respinto;

- gestione automatica dei failover: il servizio è pensato per fornire tolleranza ai guasti, per tale ragione in caso di interruzione temporanea con PostgreSQL il connettore gestisce automaticamente i tentativi di riconnessione e riprende il processamento esattamente dall'ultimo offset confermato;
- **whitelisting** dei campi: tale pratica permette di filtrare quali informazioni del messaggio Kafka devono essere memorizzate nel database, questo aspetto consente di ottimizzare l'occupazione della memoria e garantisce che siano inseriti solamente dati utili e di rilevanza analitica.

In sintesi, l'impiego di Kafka Connect permette di automatizzare il trasferimento dei dati dal messaging layer allo storage layer, con garanzie di affidabilità e persistenza.

```
curl -X POST http://localhost:8083/connectors -H "Content-Type:
application/json" -d '{
  "name": "jdbc-sink-orders",
  "config": {
    "connector.class": "io.confluent.connect.jdbc.JdbcSinkConnector",
    "connection.url": "jdbc:postgresql://postgres-db:5432/smart_warehouse",
    "connection.user": "user",
    "connection.password": "password",
    "topics": "orders",
    "table.name.format": "supplierorders",
    "insert.mode": "insert",
    "auto.create": "false",

    "value.converter": "org.apache.kafka.connect.json.JsonConverter",
    "value.converter.schemas.enable": "true",

    "pk.mode": "record_value",
    "pk.fields": "order_id",

    "fields.whitelist": "order_id,product_id,user_id,quantity"
  }
}'
```

Listato 3.1 Configurazione del connettore per i riordini merce

3.6 Dashboard Grafana

Il serving layer è affidato a Grafana, tool cardine per le attività di monitoraggio e analisi del flusso dei dati. Si è scelto di connettere la piattaforma a PostgreSQL tramite Direct SQL, per permettere la generazione di KPI e aggregazioni storiche. Questa configurazione garantisce una **visualizzazione near-real-time** dello stato del magazzino, offrendo ai gestori una dashboard dinamica per supportare le decisioni strategiche immediate. Grazie alle gestione tramite volumi Docker, l'ambiente di analisi resta isolato e persistente anche a seguito del riavvio dell'infrastruttura.

3.7 Strategia di delivery: at-least-once

Un aspetto cruciale è la gestione dei fallimenti durante il transito dei dati. Per il seguente progetto è stata scelta la semantica *at-least-once* (almeno una volta) per il flusso di elaborazione del Logic Consumer.

Le motivazioni a sostegno di questa scelta riguardano la necessità di **garantire l'integrità assoluta dell'inventario**: la perdita di un prelievo o di un deposito è un errore più grave rispetto a processarlo due volte. La semantica adottata assicura che ogni messaggio venga processato almeno una volta con successo, anche in caso di crash improvviso del sistema. Le implicazioni progettuali sono principalmente due:

1. **gestione manuale degli offset**: è stato disabilitato il commit automatico degli offset di kafka (`"enable.auto.commit": "false"`), in modo tale che **il sistema comunichi a Kafka che un messaggio è stato "letto" solamente dopo che le operazioni sul database sono state eseguite con successo**. Se il consumer dovesse, per qualsiasi ragione, arrestarsi dopo aver scritto tutte le modifiche sul database ma prima di aver confermato

l'offset a Kafka, al riavvio il messaggio sarà riprocessato garantendo che nessuna operazione resti ignorata;

2. **idempotenza delle operazioni**: il riprocessamento, però, rischia di inserire delle operazioni duplicate nel database, ciò viene risolto grazie ai vincoli di chiave primaria (**UUID**) e l'integrità transazionale di PostgreSQL. Dato che il database rifiuta l'inserimento di operazioni con stesso UUID, eventuali duplicati non altererebbero lo stato del sistema, rendendo di fatto le operazioni idempotenti.

In sintesi, la semantica adottata rappresenta un giusto compromesso tra prestazioni e affidabilità, dando fault tolerance e consistenza al sistema.

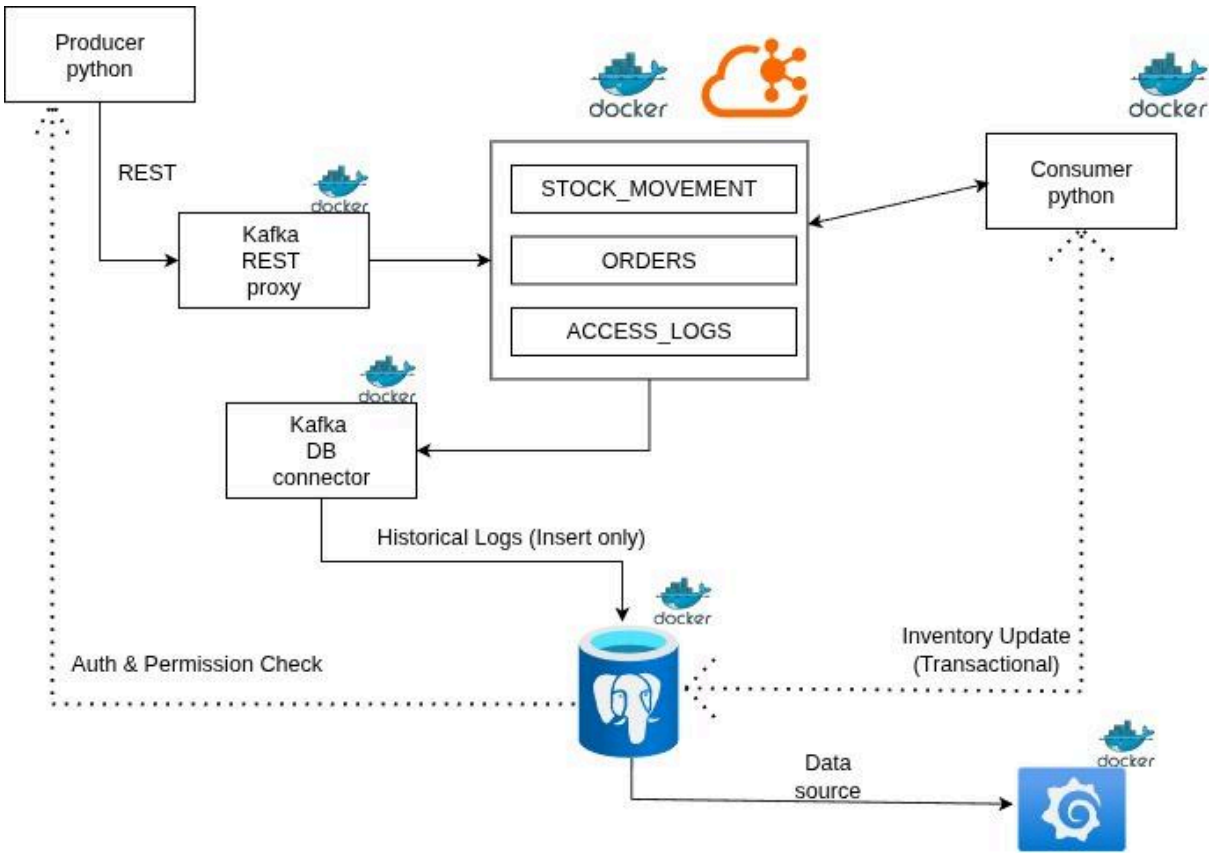

Figura 3.1 Diagramma architetturale end-to-end della pipeline

4. Modello dei dati

Le tabelle all'interno del database sono state progettate secondo una suddivisione logica in tre categorie funzionali:

- tabelle anagrafiche: informazioni statiche che danno contesto a tutte le operazioni che vengono registrate:
    - **Users**: contiene tutti gli utenti del sistema con relativo ruolo, viene interrogata per capire quali operazioni siano consentite all'utente che ha effettuato l'accesso sul palmare;
    - **Products**: contiene l'elenco di tutti i prodotti che il magazzino può gestire con soglia di riordino, viene interrogata a seguito di un prelievo per capire se è il momento di innescare un riordino automatico o meno;
    - **Sectors**: contiene l'elenco di tutti i settori del magazzino;
- tabelle di stato: costituiscono il "ponte" logico tra le informazioni statiche e la realtà del magazzino:
    - **Inventory**: rappresenta la fotografia istantanea di tutte le giacenze di magazzino, è la tabella che viene costantemente aggiornata a seguito di ogni deposito o prelievo di merce, con vincolo di integrità che impone quantità sempre maggiori o uguali di zero;
    - **UserSectorPermissions**: definisce quali utenti possono accedere a quali settori, viene interrogata quando un utente inserisce il settore al quale vuole accedere per verificare che sia effettivamente autorizzato;
- tabelle di logging & audit: alimentate automaticamente dal framework Connect, rappresentano il "diario" di tutte le operazioni che accadono all'interno del magazzino e insieme alla tabella Inventory costituiscono la fonte principale di informazioni per la costruzione dei pannelli della dashboard:

- **SupplierOrders**: contiene lo storico di tutti i riordini merce effettuati, sia manuali che automatici;
- **StockMovementsLog**: contiene lo storico di tutte le movimentazioni di prelievo e deposito;
- **AccessLogs**: contiene lo storico di tutti gli accessi effettuati ai settori di magazzino, con particolare attenzione a quelli negati.

Tutte queste tabelle sono ovviamente legate da chiavi esterne per non lasciare nessuna informazione priva di senso logico o associazione a entità reale. Invece, per le chiavi primarie la scelta è stata duplice: nelle tabelle anagrafiche è stato scelto il tipo di dato SERIAL, mentre nelle tabelle di logging è stato scelto l'UUID con lo scopo di assicurare l'idempotenza dell'esecuzione delle operazioni ed evitando così l'inserimento di duplicati.

5. Implementazione dei servizi custom

In questo capitolo viene analizzata la logica computazionale del sistema, suddivisa in due script principali: l'*handheld terminal simulator* utilizzato dagli utenti per registrare tutte le operazioni, responsabile della generazione dei dati e il *logic consumer* incaricato di ricevere e processare tutti gli eventi e di aggiornare lo stato del magazzino.

5.1 Emulazione del terminale mobile

Questo modulo rappresenta la sorgente delle informazioni che saranno poi inviate all'ingestion layer. In un contesto reale, questo script verrebbe eseguito su dispositivi mobili (come palmari o tablet industriali). Tra le sue funzionalità non troviamo solamente la generazione di informazioni, ma l'implementazione di una strategia che fornisce maggiore sicurezza, dove la validazione

dell'integrità del dato e delle autorizzazioni avviene prima che l'evento venga inoltrato al broker Kafka.

Il simulatore, dal punto di vista implementativo, adotta un approccio ibrido. Esso interagisce direttamente con PostgreSQL per le operazioni di consultazione sincrona (permessi e ruoli), inoltre per l'invio dei dati si appoggia al Kafka REST proxy. Questa scelta permette di mantenere "leggero" il client senza la necessità di importare pesanti librerie che implementano il protocollo binario nativo di Kafka, la sola libreria *requests* è sufficiente per interagire con il proxy tramite HTTP.

Al suo avvio, il simulatore richiede l'identificativo dell'utente e procede ad interrogare la tabella *Users* traendone il ruolo associato, che determinerà le funzionalità a sua disposizione nell'interfaccia testuale. Nel sistema sono predisposti **due utenti** con **operazioni** disponibili **differenti**:

- **admin** - id 1: riordino manuale e movimentazione merce in **tutti** i settori;
- **operatore** - id 2: movimentazione **solo** nei **settori 2 e 3** (carico/scarico).

Un importante controllo viene eseguito nel momento in cui gli utenti tentano di accedere fisicamente ai settori di magazzino, dunque per prelievi o depositi di merce, in tal caso il sistema implementa un design *fail-fast*: prima di impegnare risorse di rete o di calcolo per inviare messaggi al broker, viene verificata l'esistenza di un legame esplicito nella tabella *UserSectorPermissions*. Se tale query restituisce valore nullo, il simulatore interrompe l'operazione e innesca la registrazione del tentativo di accesso negato.

```
sql_permission_check = """
    SELECT 1 FROM UserSectorPermissions
    WHERE user_id = %s AND sector_id = %s; """
cursor.execute(sql_permission_check, (user_id, id_sector))
permission = cursor.fetchone()

if permission is None:
    print(f"\nERRORE: Accesso NEGATO al settore '{sectors[id_sector-1]}'.")
    send_access_log(user_id, id_sector, False)
    return (None, None, None, None, None) # Fallimento
```

Listato 5.1 Verifica permessi di accesso al settore selezionato

Una volta superati tutti i controlli e inserite le informazioni caratterizzanti dell'operazione, il simulatore procede alla costruzione dell'evento. Per consentire a Kafka Connect di memorizzare i dati in modo trasparente tramite il JDBC Sink Connector è stato necessario strutturare il payload includendo esplicitamente lo schema dei dati, grazie a questo formato self-describing il connettore mappa direttamente i tipi di dato Python verso le colonne SQL di destinazione.

```
rest_proxy_url = "http://localhost:8082/topics/stock_movements"
headers = {
    "Content-Type": "application/vnd.kafka.json.v2+json",
}

schema_definition = {
    "type": "struct",
    "fields": [
        {"type": "string", "optional": False, "field": "log_id"},
        {"type": "int32", "optional": False, "field": "product_id"},
        {"type": "int32", "optional": False, "field": "sector_id"},
        {"type": "int32", "optional": False, "field": "user_id"},
        {"type": "int32", "optional": False, "field": "quantity"}
    ],
    "optional": False,
    "name": "stockmovement.v1"
}

data_payload = {
    "log_id": str(uuid.uuid4()),
    "product_id": id_product,
    "sector_id": id_sector,
    "user_id": int(user_id),
    "quantity": quantity_to_move
}


wrapped_value = {
    "schema": schema_definition,
    "payload": data_payload
}

payload = { "records": [{"value": wrapped_value}] }

return payload, rest_proxy_url, headers, summary, id_sector
```

Listato 5.2 Costruzione del payload da inviare al REST proxy

Il ciclo di vita dell'evento viene chiuso da una richiesta POST verso l'endpoint REST proxy. La gestione della risposta HTTP garantisce un feedback immediato all'utente, che ha interagito con il sistema, riguardante la presa in carico dell'operazione.

5.2 Elaborazione degli eventi - core business logic

Questo modulo rappresenta il centro decisionale e di gestione dello stato del magazzino. Consiste in un consumer Python specializzato che opera in ascolto costante sul topic *stock_movements*, convertendo flussi di eventi asincroni in modifiche persistenti nello storage layer. L'implementazione è stata guidata da criteri di affidabilità, garantendo che ogni transazione fisica trovi corrispondenza nel database.

Per assicurare quest'ultimo aspetto è stata adottata una gestione manuale degli offset. Disabilitando il parametro *enable.auto.commit*, il software assume il controllo totale del ciclo di vita del messaggio prelevato dal topic. Il commit verso Kafka viene effettuato solo ed esclusivamente a seguito del successo della transazione SQL, è grazie a questo approccio che viene realizzata la semantica at-least-once discussa nel capitolo 3, assicurando che in caso di crash anomalo del container non ci siano movimentazioni perdute.

```
try:
    # ... logica di processamento ...
except Exception as e:
    cursor.connection.rollback() # errore, annullo operazioni
    print(f"[ERROR] Fallimento rilevato. Rollback eseguito: {e}")
else:
    # se tutto va a buon fine effettuo il commit su Postgres
    cursor.connection.commit()
    print("[DB] Transazione completata con successo.")

    # solo dopo il commit sul DB, effettuo quello su Kafka
    consumer.commit(asynchronous=False)
    print("[KAFKA] Offset confermato (At-Least-Once garantito).")
```

Listato 5.3 Gestione manuale degli offset

Compito principale del consumer è quello di aggiornare la tabella *Inventory*, con una logica differenziata sulla base della natura dell'evento:

- deposito: pattern di upsert logico, verifica la presenza del prodotto nello specifico settore e, in caso negativo, crea un nuovo record, in caso positivo effettua un incremento della rispettiva colonna quantity;
- prelievo: il decremento è eseguito con semplice logica di update con meccanismo di protezione a due livelli:
    - livello applicativo: direttamente nel palmare, prima di inoltrare l'evento al broker Kafka, viene interrogata preventivamente la tabella *Inventory* per forzare una quantità prelevata minore di quella massima disponibile, in questo modo viene impedita la generazione di messaggi logicamente errati;
    - livello fisico: nello storage layer il vincolo di integrità *CHECK (quantity >=0)* impedisce fisicamente al database di accettare transazioni che porterebbero a una giacenza negativa, garantendo allineamento tra lo stato memorizzato e la realtà fisica dei settori di magazzino.

```
if quantity_to_move > 0:
    # --- LOGICA DEPOSITO (UPSERT) ---
    sql_upsert = """
        INSERT INTO Inventory (product_id, sector_id, quantity)
        VALUES (%s, %s, %s)
        ON CONFLICT (product_id, sector_id)
        DO UPDATE SET quantity = Inventory.quantity + %s;
    """
... esecuzione query ...

elif quantity_to_move < 0:
# --- LOGICA PRELIEVO (UPDATE) ---
    sql_update = """
        UPDATE Inventory
        SET quantity = quantity + %s
        WHERE product_id = %s AND sector_id = %s
    """
... esecuzione query ...
```

Listato 5.4 Aggiornamento inventario deposito/prelievo

Come ultima feature, troviamo l'implementazione del sistema automatico di ordinazione della merce, a seguito di ogni prelievo il consumer attiva una procedura di controllo delle scorte. Una query di aggregazione calcola la somma totale delle giacenze in tutti i settori della merce appena prelevata e, nel caso in cui la somma sia al di sotto della soglia prevista (estratta dalla tabella *Products*), il consumer genera un evento JSON in modo automatico e lo inoltra al broker Kafka sul topic *orders*. Tale messaggio viene marcato con l'ID associato all'*auto_reordering_system*, ovvero 0. Grazie a questa distinzione, nella dashboard saranno distinguibili gli ordini effettuati automaticamente dal sistema e quelli effettuati manualmente dagli amministratori.

```
sql_check = """
    SELECT SUM(i.quantity), p.reorder_threshold, p.name
    FROM Inventory i
    JOIN Products p ON i.product_id = p.product_id
    WHERE i.product_id = %s
    GROUP BY p.reorder_threshold, p.name;
"""
... esecuzione query ...
if current_stock_total < threshold:
    schema_definition = {
        "type": "struct",
        "fields": [
            {"type": "string", "optional": False, "field": "order_id"},
            {"type": "int32", "optional": False, "field": "product_id"},
            {"type": "int32", "optional": False, "field": "user_id"},
            {"type": "int32", "optional": False, "field": "quantity"}
        ],
        "optional": False,
        "name": "supplierorders.v1"
    }
    data_payload = {
        "order_id": str(uuid.uuid4()),
        "product_id" : product_id,
        "user_id" : 0, # user 0 corrisponde all'auto_reorder_system
        "quantity" : 100, # quantità fissa
    }
    wrapped_value = {
        "schema": schema_definition,
        "payload": data_payload
    }
    producer.produce('orders', json.dumps(wrapped_value))
    producer.flush()
```

Listato 5.5 logica di riordino automatico

6. Orchestrazione tramite Docker Compose

L'orchestrazione di tutta l'infrastruttura è affidata a Docker Compose, che assume il ruolo di **supervisore e coordinatore di tutti i servizi** descritti nei capitoli precedenti. Questa scelta permette di non frammentare i servizi e trasforma un insieme di componenti eterogenei in un'**architettura unitaria**, facilmente attivabile e gestibile tramite semplici comandi. Attraverso questo approccio di *Infrastructure as Code*, l'intera pipeline viene definita in un unico file di configurazione con approccio dichiarativo, eliminando le dipendenze tipiche delle installazioni manuali.

Compose opera una gestione del network isolato, viene infatti creata una rete virtuale, interna e isolata all'interno del motore Docker, permettendo ai container di comunicare tra loro utilizzando i nomi dei servizi come hostname risolvibili. Inoltre, la sicurezza è migliorata dato che vengono esposti verso l'esterno solo i servizi strettamente necessari, ovvero la dashboard Grafana e il simulatore del palmare per l'immissione degli eventi, il traffico critico resta all'interno della rete protetta Docker.

Un altro compito di cui Compose si fa carico è l'**ordine di avvio** (e spegnimento) **dei servizi**; in una pipeline Big Data la sequenzialità è un requisito non trascurabile: i servizi di Ingestion o il Logic Consumer non possono operare correttamente se PostgreSQL o Kafka non sono pronti a ricevere connessioni. Questa complessità viene gestita attraverso logiche di controllo della salute dei container, assicurando che l'infrastruttura di base sia stabile e performante prima di attivare i servizi di altri layer. Grazie a queste attività di coordinamento vengono ridotte di molto le probabilità di fallimento all'avvio e a**umenta la resilienza intrinseca del sistema in caso di riavvii improvvisi**. La configurazione è centralizzata tramite l'uso di variabili globali d'ambiente, i cui valori vengono iniettati direttamente all'interno dei container al momento dello startup. Così facendo si mantiene un **unico punto di**

controllo per l'intero pipeline, semplificando le operazioni di manutenzione e scalabilità.

In sintesi, Compose non rappresenta solamente uno strumento per la distribuzione dei servizi, ma è un vero e proprio tessuto connettivo che abilita comunicazione sicura, ordinata e performante tra tutti i nodi.

7. Conclusioni e sviluppi futuri

Il progetto realizzato ha permesso di analizzare, progettare e implementare un'architettura software complessa, dedicata al **monitoraggio di tutti gli eventi riguardanti uno Smart Warehouse**. L'obiettivo principale, ovvero la realizzazione di una **pipeline Big Data resiliente e scalabile**, è stato raggiunto attraverso l'integrazione di tecnologie moderne e l'implementazione di logiche di business custom. Il sistema garantisce **corrispondenza e coerenza informativa tra la realtà fisica del magazzino e lo storage layer relazionale**.
L'impiego di Apache Kafka in modalità KRaft rappresenta il pilastro tecnologico fondamentale, confermando l'efficacia del disaccoppiamento tra produzione e consumazione degli eventi. L'adozione degli UUID univoci e la gestione manuale degli offset ha dato alla pipeline una semantica at-least-once, rendendo il sistema resiliente ai guasti. Nello stesso tempo, Docker Compose ha semplificato l'orchestrazione di tutti i servizi trasformando un ecosistema eterogeneo in un'unica unità coesa. Infine, il serving layer dove si trova la dashboard Grafana fornisce una visione trasparente su processi, giacenze e automatizzazione dei riordini.
In ottica di **evoluzioni future**, il progetto potrà accogliere i seguenti ampliamenti:

- **predictive analytics**: introduzione di modelli di Machine Learning per analizzare lo storico dei dati presenti su PostgreSQL, ottimizzando le soglie e le quantità di merce riordinate;
- deploy su **Kubernetes**: la migrazione dell'infrastruttura su un orchestratore di livello enterprise permetterebbe di gestire carichi di lavoro molto più alti, aumentando anche l'affidabilità del sistema;
- **IoT e automazione**: sensori RFID e sistemi di pesatura smart, collegati direttamente al REST proxy, porterebbe all'eliminazione della fase di

inserimento dati manuale, proiettando il magazzino verso completa automazione.

In conclusione, l'architettura proposta rappresenta un solido punto di partenza per l'implementazione di uno smart warehouse moderno, affidabile, scalabile e guidato dagli eventi.

Appendice

Vengono lasciati in allegato alla consegna del laboratorio alcuni screenshot della dashboard di monitoraggio realizzata:

- allegato 1: panoramica generale dell'intero magazzino;
- allegato 2: focus sul settore del magazzino selezionato;
- allegato 3: focus sul prodotto selezionato;
- allegato 4: selezionando l'opzione "All" dal menù a tendina viene visualizzato un confronto immediato tra giacenze attuali e livelli di soglia.